Book 2: Core Architecture & Networks

Base Layer settlement, Dual Private EVMs, Cross-EVM Bridges, Relayers, and MEV Shielding.

- Chapter 5: Base Layer Protocol & State Machine
- Chapter 6: Dual Private EVMs
- Chapter 7: Bridges & Cross-Chain Architecture
- Chapter 8: Relayers & Gas Abstraction
- Chapter 9: Sequencer Architecture & MEV Shielding

Chapter 5: Base Layer Protocol & State Machine

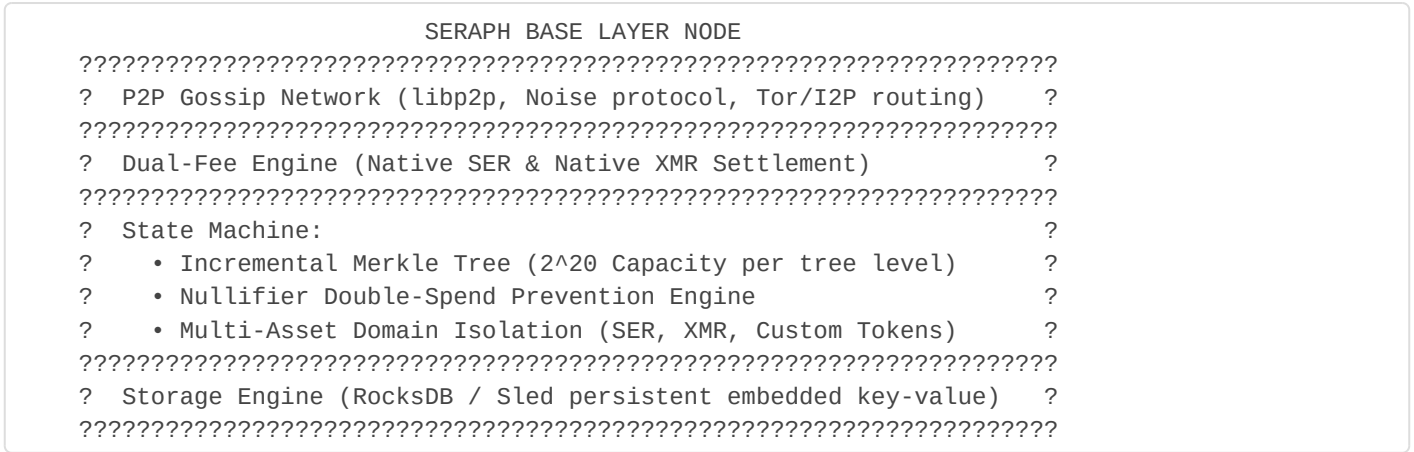
Chapter 5: Base Layer Protocol & State Machine

5.1 Protocol Architecture & Design Goals

The Seraph base layer is a custom high-performance blockchain node built in Rust (`seraph-base-layer`). It serves as the sovereign settlement substrate and shared anonymity pool for the entire Seraph ecosystem.

Unlike traditional EVM chains which record account balances directly in world state tries, the Seraph base layer maintains state purely through:

1. **The Global Commitment Tree**: An incremental Merkle tree accumulating all minted note commitments.
2. **The Nullifier Accumulator**: An append-only cryptographic registry of consumed nullifiers.



5.2 Block Structure & Consensus

Block Header Specification

Every block header on the Seraph base layer contains:

- `version`: Protocol version integer (32-bit).
- `previous_block_hash`: 32-byte cryptographic hash of parent block.
- `merkle_root`: Root of the transaction inclusion Merkle tree.
- `state_root`: Root of the active note commitment tree after processing all transactions.
- `nullifier_root`: Root of the newly consumed nullifiers batch.
- `timestamp`: 64-bit UTC UNIX timestamp.
- `height`: Monotonically increasing block index.
- `nonce`: Proof-of-work / validator signature depending on network phase.

Consensus Evolution

- **Phase 0 & 1 (Devnet & Testnet)**: Authority-validated sequencing with deterministic consensus for rapid testing and low latency.
- **Phase 2 & Beyond**: Transition to a hybrid Proof-of-Stake / RandomX-variant checkpointing engine, allowing native Monero miners to co-mine or validate Seraph blocks trustlessly.

5.3 Transactions and Enotes

In the Seraph base layer, transactions are strictly balance-conserving transitions consuming input notes and producing new output notes:

```
// Core Base-Layer Transaction Representation (base-layer/src/ledger.rs)
pub struct Transaction {
    pub inputs: Vec<[u8; 32]>,       // Spent note nullifiers
    pub outputs: Vec<Note>,          // New confidential notes
    pub fee_ser: u64,                // Fee paid in SER units
    pub fee_xmr: u64,                // Fee paid in atomic XMR units
    pub public_inputs: Vec<u8>,      // Merkle roots and public inputs
    pub proof: Vec<u8>,              // Cryptographic ZK spend proof
}
```

Transaction Validation Rules:

1. **Nullifier Uniqueness**: Every input nullifier must not currently exist in the ledger's `nullifiers` set: $$\forall \mathcal{N}_i \in \text{inputs}: \mathcal{N}_i \notin \text{Ledger.nullifiers}$$
2. **Atomic Nullifier Consumption**: All input nullifiers are inserted into `Ledger.nullifiers` within an atomic database transaction. If any nullifier fails, the entire transaction reverts.
3. **Commitment Insertion**: All output commitments $C_j = \text{output}_j.\text{commitment}()$ are inserted sequentially into the commitment tree.
4. **Conservation of Value**: The sum of inputs must equal the sum of outputs plus specified fees across each distinct asset class.

5.4 Dual-Fee Engine: Native SER and Native XMR

A signature innovation of the Seraph base layer is its native **dual-currency fee engine**. Transactions can pay inclusion fees in either:

1. **SER (Native Protocol Token)**: Supporting deflationary burning and treasury allocation.
2. **XMR (Native Monero)**: Bridged or natively integrated, allowing Monero holders to settle transactions directly without holding secondary gas tokens.

Fee Settlement Flow:

- When a user submits a transaction specifying `fee_xmr`, the transaction inputs consume atomic XMR enotes.
- Relayers or block proposers can receive fees in XMR directly into their private stealth addresses.
- Multi-asset isolation ensures that XMR and SER amounts never mix or cause rounding errors.

5.5 Multi-Asset Domain Isolation

Seraph avoids token cross-contamination by binding the `AssetId` directly into the leaf commitment hash:

```
// base-layer/src/assets.rs
pub enum AssetId {
    Seraph = 0,
    Xmr = 1,
    Custom(u32),
}

impl Note {
    pub fn commitment(&self) -> [u8; 32] {
        let mut hasher = Keccak256::new();
        hasher.update(self.asset.domain_tag());
        hasher.update(self.amount.to_be_bytes());
        hasher.update(self.recipient_pubkey);
        hasher.update(self.blinding);
        hasher.finalize().into()
    }
}
```

Even if an attacker crafts an XMR note with identical amount, recipient key, and blinding scalar as a SER note, the resulting commitments and nullifiers are cryptographically isolated.

5.6 Networking & P2P Topology

The networking stack scales through three distinct milestones:

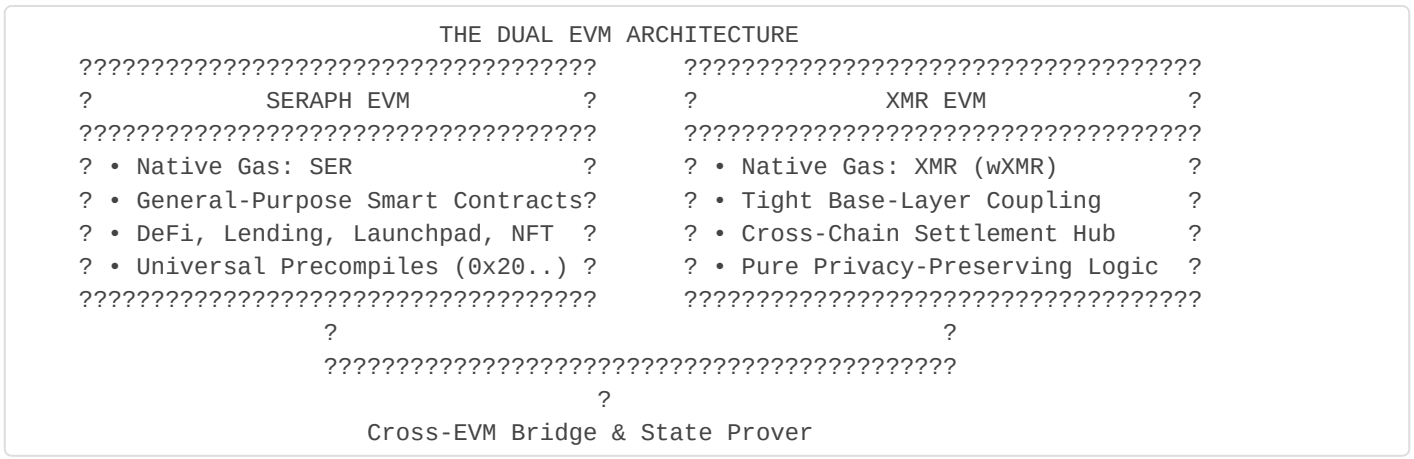
1. **Phase 0 (In-Process PoC)**: Validated in Rust via multi-actor test harnesses (`tests/adversarial_pentest.rs`) confirming state transitions, mempool double-spend rejections, and ring buffer evictions.
2. **Phase 1 (Single-Node Devnet)**: Local RPC node exposing JSON-RPC over HTTP and WebSocket, integrated with the `@seraph-protocol/sdk`.
3. **Phase 2 (Multi-Node Testnet)**: Fully decentralized libp2p gossip mesh with:
   - Dandelion++ transaction dissemination to prevent IP address triangulation.
   - Built-in Tor and I2P daemon proxies for complete network-layer anonymity.

Chapter 6: Dual Private EVMs

Chapter 6: Dual Private EVMs

6.1 Architectural Overview

Seraph deploys two specialized Ethereum Virtual Machines designed to provide programmable privacy without sacrificing developer tooling:



6.2 Seraph EVM: General-Purpose Confidential Execution

The **Seraph EVM** is an optimized EVM runtime where:

- **Base Fee Token**: Paid natively in SER tokens.
- **Contract Environment**: 100% bytecode compatible with EVM bytecode (Cancun/Prague), allowing existing Solidity, Vyper, and Yul contracts to deploy seamlessly.
- **Confidential Extensions**: Developers can import Seraph privacy libraries to shield variables, store encrypted state in storage slots, and authenticate callers via zero-knowledge proofs.

6.3 XMR EVM: Native XMR Settlement

The **XMR EVM** is a purpose-built sister EVM where:

- **Gas Settlement in XMR**: Gas fees are calculated in gas units but deducted from user balances in atomic XMR (piconeros) or bridged `wXMR`.
- **Base-Layer Synchronization**: The state of the XMR EVM is closely coupled to the base layer's Monero anonymity pool.
- **Cross-EVM Interoperability**: Contracts on the XMR EVM can trigger cross-contract calls into Seraph EVM via the Cross-EVM Bridge.

6.4 Custom Privacy Precompiles

Both EVM runtimes introduce specialized cryptographic precompiles located in the reserved precompile address range (`0x20` to `0x25`):

Address	Precompile Name	Description	Gas Cost
`0x20`	`PEDERSEN_COMMIT`	Computes $C = vH + rG$ over BN254	750 gas
`0x21`	`POSEIDON_HASH`	High-throughput algebraic hash for ZK circuits	300 gas
`0x22`	`STEALTH_DERIVE`	DKSAP derivation $P = \mathcal{H}_s(rA)G + B$	1,200 gas
`0x23`	`BULLETPROOF_VERIFY`	Verifies range proofs $v \in [0, 2^{64}-1]$	12,000 gas
`0x24`	`MERKLE_INSERT`	O(1) incremental Merkle tree insertion	2,500 gas
`0x25`	`NULLIFIER_CHECK`	Checks and consumes nullifiers in state	800 gas

These precompiles reduce the computational cost of zero-knowledge privacy operations by up to 90% compared to equivalent pure-Solidity implementations.

6.5 Confidential Contract State & Encrypted Variables

Traditional EVMs make all contract storage slots publicly readable by anyone running a node. Seraph introduces **Confidential Storage Slots**:

```
// Example: Confidential Storage in Solidity
contract PrivateBalanceVault {
    // Encrypted mapping: slot contains encrypted ciphertext + commitment
    mapping(address => bytes32) private encryptedBalances;

    // Homomorphic update via precompile
    function transferConfidential(
        bytes32 commitmentIn,
        bytes32 commitmentOut,
        bytes calldata zkProof
```

```
    ) external {
        require(IVerifier(0x23).verifyProof(zkProof, ...), "Invalid proof");
        // State updated without exposing raw balance
    }
}
```

- Data stored in confidential slots is encrypted with a contract-level ephemeral key.
- State modifications require valid zero-knowledge proofs of authorization.
- Public observers see state transitions occur, but cannot decode the underlying quantities.

6.6 Tooling & Developer Compatibility

Seraph maintains strict compatibility with standard Web3 development tools:

- **Hardhat & Foundry**: Compile and test using standard `forge build` and `forge test` without custom compiler forks.
- **Ethers.js & Viem**: Standard JSON-RPC interface support (`eth_call`, `eth_sendRawTransaction`, `eth_getLogs`).
- **MetaMask & Web3 Wallets**: Connect directly by adding RPC URL and Chain ID.
- **OpenZeppelin Contracts**: Inherit directly from standard OpenZeppelin v5 libraries (`ERC20`, `ReentrancyGuard`, `Ownable`).

Chapter 7: Bridges & Cross-Chain Architecture

Chapter 7: Bridges & Cross-Chain Architecture

7.1 Cross-Chain Design Principles

The Seraph cross-chain architecture connects three heterogeneous execution environments:

1. **The Sovereign Seraph Base Layer** (Rust settlement & shared anonymity pool).
2. **Seraph EVM** (General-purpose EVM, gas in SER).
3. **XMR EVM** (Specialized EVM, gas in XMR/wXMR).

Rather than relying on trusted multisig federation bridges, Seraph implements an **optimistic, fraud-proof verified, nullifier-tagged bridge** (`CrossEVMBridge.sol`).



7.2 The Lock, Prove & Mint Mechanism

Cross-chain asset transfers follow a strict three-stage protocol:

Step 1: Lock on Source Chain

A user invokes `bridgeOut` on the source bridge:

```
function bridgeOut(
    address recipient,
    uint256 amount,
    bytes32 destChainId
) external nonReentrant returns (uint256 nonce);
```

- Tokens are locked into the bridge contract.
- A unique `outboundNonce` is assigned.
- An event `TokensLocked(nonce, sender, recipient, amount, destChainId)` is emitted.

Step 2: Optimistic Batch Submission

A bonded relayer packages outbound bridge transactions into a Merkle state root and submits it to the destination bridge:

```
function submitStateBatch(
    bytes32 sourceChainId,
    bytes32 stateRoot
) external whenNotPaused returns (uint256 batchId);
```

- Requires a minimum relayer bond of **1.0 ETH** (`MIN_RELAYER_BOND = 1 ether`).
- Timestamp is recorded, opening the **1-hour challenge window** (`CHALLENGE_WINDOW = 1 hours`).

Step 3: Withdrawal Claim

After the 1-hour window expires without challenge, the recipient calls `claimBridged`:

```
function claimBridged(
    uint256 batchId,
    bytes32 nullifier,
    address recipient,
    uint256 amount,
    bytes32[] calldata merkleProof
) external nonReentrant whenNotPaused;
```

- The bridge verifies the leaf: $$\text{leaf} = \text{Keccak256}(\text{nullifier} \mathbin{\Vert} \text{recipient} \mathbin{\Vert} \text{amount} \mathbin{\Vert} \text{localChainId})$$
- The bridge checks `merkleProof` against `batches[batchId].stateRoot`.
- The nullifier is recorded in `processedNullifiers[nullifier] = true`, preventing replay attacks.
- Tokens are unlocked or minted to the recipient.

7.3 Fraud Proofs, Bonds & Slashing

The bridge security model is economically protected through bonded collateral:

1. **Bond Requirements**: Any relayer wishing to submit batches must bond at least 1.0 ETH via `bondRelayer()`.
2. **Unbonding Locks**: When a relayer calls `unbondRelayer(amount)`, the contract verifies that no batches submitted by that relayer are currently inside the active 1-hour challenge window.
3. **Slashing Fraudulent Batches**: If a relayer submits an invalid root containing fictitious deposits, the protocol Guardian (or any automated watcher) invokes:

   ```
   function challengeBatch(uint256 batchId, address challenger) external nonReentrant;
   ```

   - The batch is immediately marked `challenged = true`, permanently preventing any token claims from that batch.
   - The relayer's entire bonded collateral is slashed to 0.
   - Slashed ETH is paid directly to the challenger as an economic bounty.

7.4 Emergency Pause Circuit Breakers

To protect user funds against zero-day vulnerabilities or upstream chain reorganizations, the bridge incorporates an emergency pause mechanism:

```
function setPaused(bool _paused) external {
    if (msg.sender != guardian) revert Unauthorized();
    paused = _paused;
    if (_paused) emit Paused(msg.sender);
    else emit Unpaused(msg.sender);
}
```

When paused:

- `bridgeOut()` reverts with `BridgePaused()`.
- `submitStateBatch()` reverts with `BridgePaused()`.
- `claimBridged()` reverts with `BridgePaused()`.
- Relayer bonds remain safely locked in escrow until the protocol guardian completes security review and unpauses.

7.5 Bridged XMR Representation (`wXMR`)

To represent native Monero on EVM layers:

- Monero locked in base-layer stealth multisig vaults is mirrored as an ERC-20 compliant `wXMR` token on Seraph EVM and XMR EVM.
- `wXMR` conforms to standard 18-decimal precision.
- Can be utilized across DeFi lending markets, automated liquidity pools, and gas payments via `GasRelayer`.

Chapter 8: Relayers & Gas Abstraction

Chapter 8: Relayers & Gas Abstraction

8.1 The Gas Dilemma in Confidential Systems

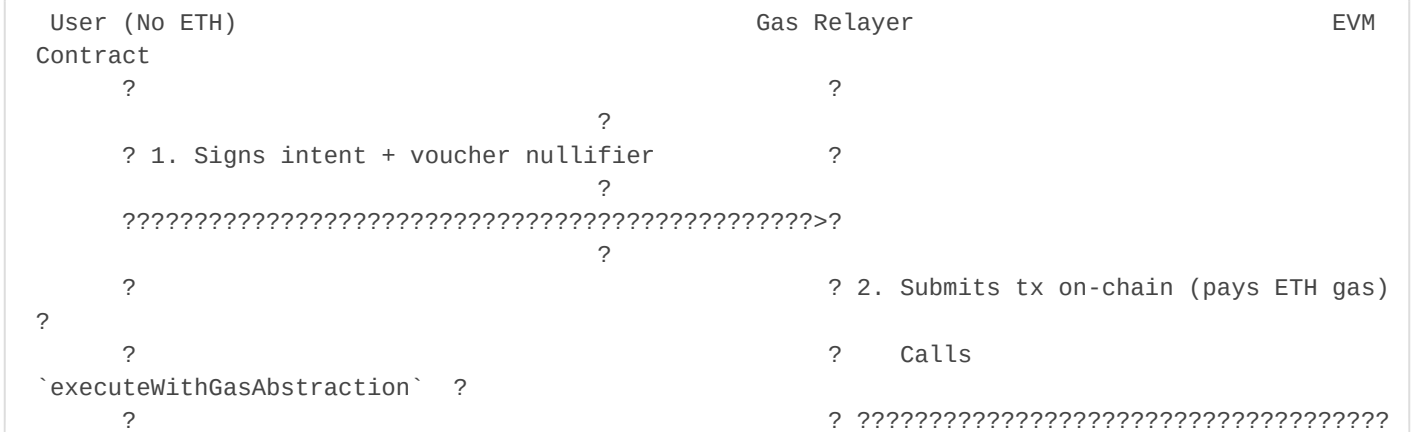
In traditional Web3 applications, interacting with a blockchain requires the user to hold native gas tokens (such as ETH) in their public wallet. This creates a severe privacy leak:

- Even if a user receives funds anonymously into a private stealth address, they cannot move those funds without funding that stealth address with gas.
- The gas funding transaction (originating from a CEX or public wallet) immediately establishes a graph-analysis link to the private address, completely destroying privacy.

Seraph eliminates this vulnerability through **account-abstracted gas relayers** (`GasRelayer.sol`).

8.2 The GasRelayer Architecture

The `GasRelayer` contract allows users to execute arbitrary EVM transactions without holding any native ETH in their executing wallet. Instead, transaction fees are settled using **pre-funded gas vouchers** in SER or XMR:



```
>?
      ?                                                    ?
                                      ? 3. Executes user payload
      ?                                                    ? <
????????????????????????????????????????
      ?                                                    ? 4. Reimburses relayer in SER or
XMR   ?
      ?                                                    ?    from voucher pool
balance          ?
```

8.3 Voucher Funding & Nullifier Settlement

1. Funding a Gas Voucher

A user (or onboarding sponsor) deposits fee tokens into the voucher pool:

```
function fundGasVoucher(
    address feeToken,
    uint256 amount,
    bytes32 voucherCommitment
) external nonReentrant;
```

- `feeToken`: SER or `wXMR`.
- `voucherCommitment`: A cryptographic hash committing to an unspent secret scalar.
- Tokens are held in the contract's pooled liquidity escrow.

2. Gas-Abstracted Execution

When the user desires to execute a call:

```
function executeWithGasAbstraction(
    bytes32 voucherNullifier,
    address feeToken,
    uint256 feeAmount,
    address target,
    bytes calldata data
) external nonReentrant returns (bytes memory result);
```

- The relayer broadcasts the transaction and pays the native gas in ETH.
- The contract verifies that `voucherNullifier` has not been used (`!usedVouchers[voucherNullifier]`).
- The contract verifies that `voucherPoolBalance[feeToken] >= feeAmount`.
- The contract marks `usedVouchers[voucherNullifier] = true`.
- The contract transfers `feeAmount` of `feeToken` directly to the relayer as compensation.
- The contract executes `target.call(data)` on behalf of the user.

8.4 Defensive Security & Target Filtering

To prevent malicious users from crafting exploit calls that drain protocol assets or hijack the relayer, `GasRelayer.sol` implements strict target validation:

```
error ForbiddenTarget();

if (
    target == address(0) ||
    target == address(this) ||
    target == address(serToken) ||
    target == address(wxmrToken)
) revert ForbiddenTarget();
```

Attack Mitigation:

- An attacker cannot supply `target = address(serToken)` with `data = transfer(...)` to drain tokens held by the relayer pool.
- An attacker cannot call internal admin functions on `GasRelayer` itself.
- Reentrancy locks (`nonReentrant`) prevent recursive callback drains.

8.5 Gas Rate Oracles & Fee Adjustment

The conversion rate between native ETH gas and fee tokens (SER / XMR) is dynamically adjusted by governance:

```
function setTokenGasRate(address feeToken, uint256 rate) external onlyOwner;
```

This ensures that relayers remain profitable across market volatility while users pay minimal overhead (typically < 3% over raw gas costs).

Chapter 9: Sequencer Architecture & MEV Shielding

Chapter 9: Sequencer Architecture & MEV Shielding

9.1 The MEV Vulnerability in Transparent Blockchains

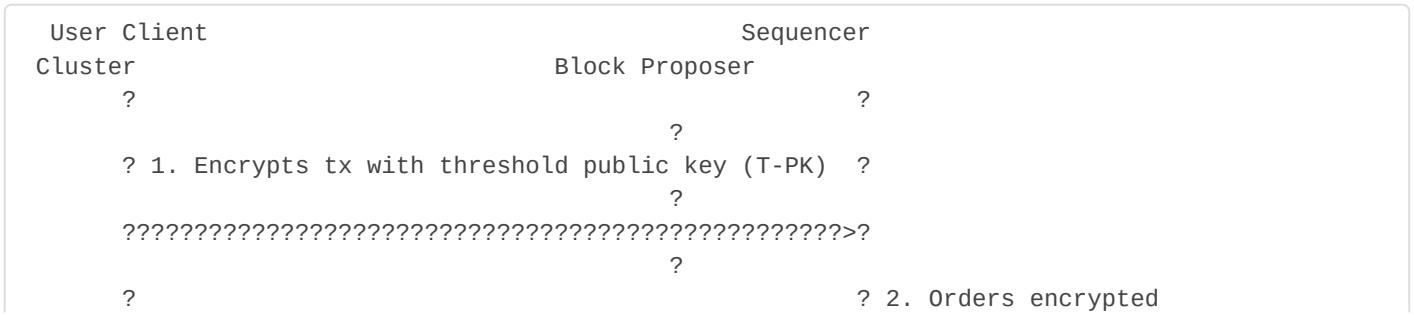
On public smart contract networks, transactions wait in a transparent memory pool (mempool) prior to block inclusion. This allows predatory searchers and maximal extractable value (MEV) bots to:

1. **Front-Run**: Detect profitable DEX trades and insert their own transaction ahead by paying higher priority gas.
2. **Sandwich Attack**: Enclose a user's swap between a buy and sell transaction, intentionally extracting slippage from ordinary users.
3. **Censorship & Liquidation Sniping**: Reorder transactions to trigger liquidations or censor privacy transactions.

In a privacy protocol, MEV poses an existential threat: if transaction ordering or metadata leaks before execution, privacy guarantees can be severely compromised.

9.2 Encrypted Mempool Architecture

Seraph deploys an **Encrypted Mempool with Threshold Cryptography** to eliminate front-running and toxic MEV:





Protocol Steps:

1. **Client-Side Encryption**: When a user constructs a transaction, the payload (recipient, function selector, parameters, value) is encrypted using a distributed **Threshold Public Key (PK_{thresh})**: $$C_{\text{tx}} = \text{Encrypt}(PK_{\text{thresh}}, \text{tx})$$ Only the gas bid and sender's zero-knowledge validity proof are visible to relayers.
2. **Order Commitment**: Sequencer nodes receive C_{tx} and commit to an immutable ordered sequence: $$H_{\text{batch}} = \text{Keccak256}(C_1 \mathbin{\Vert} C_2 \mathbin{\Vert} \dots \mathbin{\Vert} C_k)$$ Once H_{batch} is posted to the base layer, the transaction order is mathematically locked and cannot be altered.
3. **Threshold Decryption**: A distributed committee of n validator nodes each release an ephemeral decryption key share. Once t of n shares (e.g., 5-of-7 quorum) are broadcast, the full batch is decrypted simultaneously.
4. **Deterministic Execution**: The decrypted transactions are executed in the exact pre-committed order. Because searchers cannot see trade parameters before the order is fixed, front-running and sandwiching are mathematically impossible.

9.3 Protection Against Toxic MEV

Attack Vector	Vulnerability on Public EVMs	Mitigation on Seraph Dual EVMs
DEX Sandwich Attacks	Searchers front-run & back-run slippage	**Impossible**: Swap amounts and paths are hidden inside threshold ciphertext until execution order is locked.
Liquidation Front-Running	Bots compete for liquidation priority gas	**Fair Ordering**: First-seen encrypted batches are executed sequentially without priority auction bidding.

Attack Vector	Vulnerability on Public EVMs	Mitigation on Seraph Dual EVMs
Statistical Arbitrage	Bots front-run oracle price updates	**Time-locked Batches**: Price updates and user transactions settle within atomic execution blocks.
Censorship	Validators selectively exclude privacy txs	**Zero Metadata**: Validators cannot distinguish between privacy pool deposits, transfers, and standard swaps.

9.4 Decentralized Sequencer Rotation & Fault Recovery

To prevent single-point-of-failure centralization, Seraph implements a **Decentralized Sequencer Set (DSS)**:

1. **Epoch-Based Rotation**: A new primary sequencer is elected deterministically every 100 blocks based on bonded SER stake.
2. **Slashing for Non-Liveness**: If a sequencer fails to submit a batch within the 2-second block window, the standby backup sequencer immediately takes over, and the inactive sequencer's stake is slashed.
3. **Equivocation Slashing**: If a sequencer signs two conflicting batch orders for the same block height, an automated fraud proof slashes 100% of their bonded stake.